a page. This might happen in a community forum, for example, where user comments are stored and retrieved from a database. Anyone who visits the page subsequently will cause the malicious script to execute.

In previous versions of ASP.NET, the default mitigation against XSS attacks was a feature known as *request validation*, which screened user input for potentially dangerous characters, like the angle brackets used to delimit HTML script tags: < and >. If the query string or a form field included blacklisted characters, an `HttpRequestValidationException` was raised, and the application crashed. As a developer, you had to opt out of this feature if you wanted to be able to accept angle brackets and so on from users. This feature was not included in ASP.NET Core on the basis that the ASP.NET team felt that it should be up to the developer of an application to decide what is valid and what isn't. Nowadays, the default mitigation against XSS is to have Razor automatically encode all output as HTML, as discussed in section 3.1.5.

There will be occasions in which the application relies on HTML being posted, stored in a database, and later retrieved for inclusion within the markup of a rendered page, using the `Html.Raw` method. Typically, you will use a rich text editor library, such as TinyMCE (https://www.tiny.cloud) or CKEditor (https://ckeditor.com/), to generate the markup. You should check their documentation for instructions on how to create a whitelist of permissible HTML tags, but bear in mind that this validation will take place on the client and cannot be relied on. You should also validate the submitted input on the server.

So how do you validate the input to prevent, for example, script tags from being included in posted submissions? You could use simple string methods like `IndexOf` or `Contains` to check for the presence of the dangerous string or possibly even regular expressions. My preference is to use AngleSharp (https://github.com/AngleSharp/AngleSharp), a great open source project that enables you to parse fragments of HTML and query it using CSS-like selectors to easily determine whether disallowed tags have been included in the posted fragment. To illustrate this, assume the `City` class has a `Description` property, where the user can provide some information about the city in question, and they can do this with HTML. In this case, you add a public property to the `CreateModel` class in the CityManager folder and decorate it with the `BindProperty` attribute:

```
[BindProperty]
public string Description { get; set; }
```